Arcade

Generated by Doxygen 1.16.1

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 4

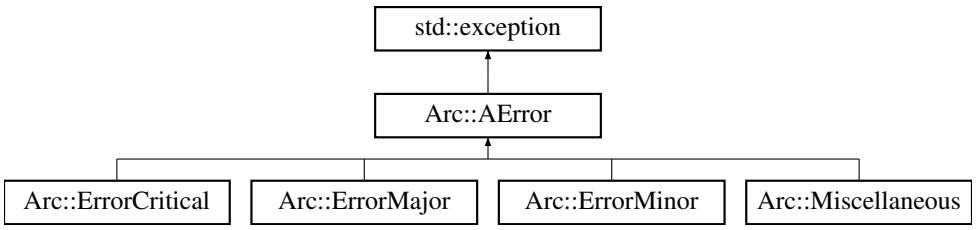
Class Documentation

4.1 Arc::AError Class Reference

Base exception class for all Arcade system exceptions.

`#include <ArcError.hpp>`

Inheritance diagram for Arc::AError:



Public Member Functions

- AError (std::string message)

  Creates an error with a message.
- const char * what () const noexcept override

  Returns the error message.

4.1.1 Detailed Description

Base exception class for all Arcade system exceptions.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 AError()

```
Arc::AError::AError (
            std::string message)  [inline]
```

Creates an error with a message.

Parameters

message	Error description

4.1.3 Member Function Documentation

4.1.3.1 what()

```
const char * Arc::AError::what () const  [inline], [override], [noexcept]
```

Returns the error message.

Returns

C-string describing the error

The documentation for this class was generated from the following file:

- src/Include/ArcError.hpp

4.2 Arc::Core Class Reference

Main controller for the Arcade game launcher system.

```
#include <Core.hpp>
```

Public Member Functions

- Core (const std::string &defaultRendererLib, const std::vector< std::pair< std::string, std::string > > &rendererLibs, const std::vector< std::pair< std::string, std::string > > &gameLibs)

  Initializes the Core with available libraries.
- ~**Core** ()

  Destructor - cleans up loaded libraries.
- void **run** ()

  Starts the main event loop.
- void launchGame (const std::string &gamePath, const std::string &rendererPath)

  Launches a game with a specific renderer.

Static Public Member Functions

- static std::optional< LibraryMetadata > extractLibraryMetadata (const std::string &libPath)

  Extracts metadata from a library file.
- static void validateDefaultRenderer (const std::string &rendererPath)

  Validates that a library is a valid renderer.

Public Attributes

- bool **_is_running**
- int **_current_game_index** = 0
- int **_current_renderer_index** = 0
- std::string **_username**

4.2.1 Detailed Description

Main controller for the Arcade game launcher system.

Manages game launching, renderer switching, menu display, and dynamic library loading and validation.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Core()

```
Arc::Core::Core (
            const std::string & defaultRendererLib,
            const std::vector< std::pair< std::string, std::string > > & rendererLibs,
            const std::vector< std::pair< std::string, std::string > > & gameLibs)
```

Initializes the Core with available libraries.

Parameters

defaultRendererLib	Path to the default renderer library
rendererLibs	Vector of available renderer libraries
gameLibs	Vector of available game libraries

4.2.3 Member Function Documentation

4.2.3.1 extractLibraryMetadata()

```
std::optional< Arc::LibraryMetadata > Arc::Core::extractLibraryMetadata (
            const std::string & libPath)  [static]
```

Extracts metadata from a library file.

Parameters

libPath	Path to the library to examine

Returns

Optional metadata if valid, nullopt otherwise

4.2.3.2 launchGame()

```
void Arc::Core::launchGame (
            const std::string & gamePath,
            const std::string & rendererPath)
```

Launches a game with a specific renderer.

Parameters

gamePath	Path to the game library
rendererPath	Path to the renderer library

4.2.3.3 validateDefaultRenderer()

```
void Arc::Core::validateDefaultRenderer (
            const std::string & rendererPath)  [static]
```

Validates that a library is a valid renderer.

Parameters

rendererPath	Path to the renderer library

Exceptions

ErrorCritical	if validation fails

The documentation for this class was generated from the following files:

- src/Core/Core.hpp
- src/Core/Core.cpp

4.3 Arc::DLloader< T > Class Template Reference

Template class for loading dynamic libraries at runtime.

`#include <DLloader.hpp>`

Public Member Functions

- DLloader (const std::string &libPath)
  Loads a dynamic library and resolves the entryPoint symbol.
- ~DLloader ()
  Destructor - does not unload the library.
- T * getInstance (std::map< std::string, Item > &itemList)
  Creates an instance using the loaded library's entryPoint.

4.3.1 Detailed Description

template<typename T>
class Arc::DLloader< T >

Template class for loading dynamic libraries at runtime.

Handles opening shared libraries (.so files) and retrieving the entryPoint function to create instances of game or renderer implementations.

Template Parameters

T	The interface type to instantiate (IGame or IRenderer)

4.3.2 Constructor & Destructor Documentation

4.3.2.1 DLloader()

```
template<typename T>
Arc::DLloader< T >::DLloader (
            const std::string & libPath)  [inline]
```

Loads a dynamic library and resolves the entryPoint symbol.

Parameters

libPath	Path to the shared library file

4.3.2.2 ~DLloader()

```
template<typename T>
Arc::DLloader< T >::~DLloader ()  [inline]
```

Destructor - does not unload the library.

Libraries remain loaded for the application lifetime to keep instances valid.

4.3.3 Member Function Documentation

4.3.3.1 getInstance()

```
template<typename T>
T * Arc::DLloader< T >::getInstance (
            std::map< std::string, Item > & itemList)  [inline]
```

Creates an instance using the loaded library's entryPoint.

Parameters

itemList	Item map to pass to the entryPoint

Returns

Pointer to the created instance, or nullptr if loading failed

The documentation for this class was generated from the following file:

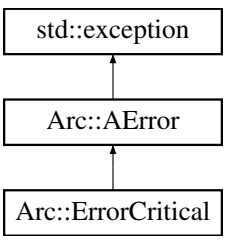
- src/Include/DLloader.hpp

4.4 Arc::ErrorCritical Class Reference

Exception for critical, fatal errors.

```
#include <ArcError.hpp>
```

Inheritance diagram for Arc::ErrorCritical:



Public Member Functions

- ErrorCritical (std::string message)
  Creates a critical error.

Public Member Functions inherited from Arc::AError

- AError (std::string message)
  Creates an error with a message.
- const char * what () const noexcept override
  Returns the error message.

4.4.1 Detailed Description

Exception for critical, fatal errors.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 ErrorCritical()

```
Arc::ErrorCritical::ErrorCritical (
            std::string message)  [inline]
```

Creates a critical error.

Parameters

message	Error description

The documentation for this class was generated from the following file:

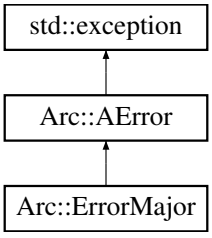
- src/Include/ArcError.hpp

4.5 Arc::ErrorMajor Class Reference

Exception for major errors affecting functionality.

`#include <ArcError.hpp>`

Inheritance diagram for Arc::ErrorMajor:



Public Member Functions

- ErrorMajor (std::string message)
  Creates a major error.

Public Member Functions inherited from Arc::AError

- AError (std::string message)
  Creates an error with a message.
- const char * what () const noexcept override
  Returns the error message.

4.5.1 Detailed Description

Exception for major errors affecting functionality.

4.5.2 Constructor & Destructor Documentation

4.5.2.1 ErrorMajor()

```
Arc::ErrorMajor::ErrorMajor (
            std::string message)  [inline]
```

Creates a major error.

Parameters

message	Error description

The documentation for this class was generated from the following file:

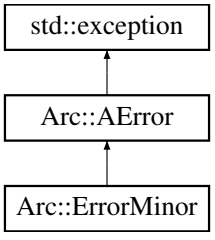
- src/Include/ArcError.hpp

4.6 Arc::ErrorMinor Class Reference

Exception for minor, recoverable errors.

`#include <ArcError.hpp>`

Inheritance diagram for Arc::ErrorMinor:



Public Member Functions

- ErrorMinor (std::string message)
  Creates a minor error.

Public Member Functions inherited from Arc::AError

- AError (std::string message)
  Creates an error with a message.
- const char * what () const noexcept override
  Returns the error message.

4.6.1 Detailed Description

Exception for minor, recoverable errors.

4.6.2 Constructor & Destructor Documentation

4.6.2.1 ErrorMinor()

```
Arc::ErrorMinor::ErrorMinor (
            std::string message)  [inline]
```

Creates a minor error.

Parameters

message	Error description

The documentation for this class was generated from the following file:

- src/Include/ArcError.hpp

4.7 Arc::Event Struct Reference

Represents a user input event.

```
#include <Event.hpp>
```

Public Attributes

- Type **type**
- Key **code**
- Mouse **mouse**

4.7.1 Detailed Description

Represents a user input event.

The documentation for this struct was generated from the following file:

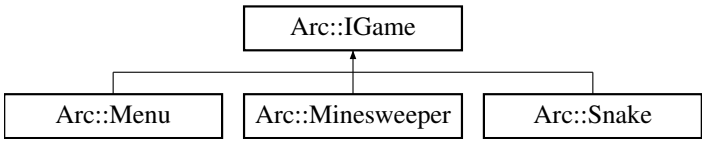
- src/Include/Event.hpp

4.8 Arc::IGame Interface Reference

Abstract interface for game implementations in the Arcade system.

`#include <IGame.hpp>`

Inheritance diagram for Arc::IGame:



Public Member Functions

- virtual void update (Event event)=0
  Updates the game state based on the received event.
- virtual void handleKey (Event event)=0
  Handles keyboard inputs for the game.
- virtual std::vector< int > getScore () const =0
  Retrieves the current game score.
- virtual bool isGameOver () const =0
  Checks if the game is over.

4.8.1 Detailed Description

Abstract interface for game implementations in the Arcade system.

All game plugins must inherit from this interface and implement these pure virtual methods to be compatible with the Arcade launcher.

4.8.2 Member Function Documentation

4.8.2.1 getScore()

```
virtual std::vector< int > Arc::IGame::getScore () const  [pure virtual]
```

Retrieves the current game score.

Returns

Vector of integers representing the score(s)

Implemented in Arc::Menu, Arc::Minesweeper, and Arc::Snake.

4.8.2.2 handleKey()

```
virtual void Arc::IGame::handleKey (
            Event event)  [pure virtual]
```

Handles keyboard inputs for the game.

Parameters

event	The keyboard event to handle

Implemented in Arc::Menu, Arc::Minesweeper, and Arc::Snake.

4.8.2.3 isGameOver()

```
virtual bool Arc::IGame::isGameOver () const  [pure virtual]
```

Checks if the game is over.

Returns

true if the game is over, false otherwise

Implemented in Arc::Menu, Arc::Minesweeper, and Arc::Snake.

4.8.2.4 update()

```
virtual void Arc::IGame::update (
            Event event)  [pure virtual]
```

Updates the game state based on the received event.

Parameters

event	The event to process (key press, mouse, etc.)

Implemented in Arc::Menu, Arc::Minesweeper, and Arc::Snake.

The documentation for this interface was generated from the following file:

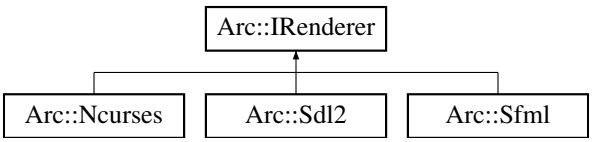
- src/Include/IGame.hpp

4.9 Arc::IRenderer Interface Reference

Abstract interface for graphics renderer implementations.

`#include <IRenderer.hpp>`

Inheritance diagram for Arc::IRenderer:



Public Member Functions

- virtual void display (std::map< std::string, Item > &items)=0
  Renders all items on the display.
- virtual void createWindow (int width, int height, const std::string &title)=0
  Creates the display window.
- virtual bool pollEvent (Event &event)=0
  Polls for user input events.

4.9.1 Detailed Description

Abstract interface for graphics renderer implementations.

All renderer plugins must inherit from this interface and implement these methods to be compatible with the Arcade launcher.

4.9.2 Member Function Documentation

4.9.2.1 createWindow()

```
virtual void Arc::IRenderer::createWindow (
            int width,
            int height,
            const std::string & title)  [pure virtual]
```

Creates the display window.

Parameters

width	Window width in pixels
height	Window height in pixels
title	Window title

Implemented in Arc::Ncurses, Arc::Sdl2, and Arc::Sfml.

4.9.2.2 display()

```
virtual void Arc::IRenderer::display (
            std::map< std::string, Item > & items)  [pure virtual]
```

Renders all items on the display.

Parameters

items	Map of named items to render

Implemented in Arc::Ncurses, Arc::Sdl2, and Arc::Sfml.

4.9.2.3 pollEvent()

```
virtual bool Arc::IRenderer::pollEvent (
            Event & event)  [pure virtual]
```

Polls for user input events.

Parameters

event	Output parameter populated with event data if available

Returns

true if an event was received, false otherwise

Implemented in Arc::Ncurses, Arc::Sdl2, and Arc::Sfml.

The documentation for this interface was generated from the following file:

- src/Include/IRenderer.hpp

4.10 Arc::Item Struct Reference

Represents a renderable object in the display.

```
#include <Utils.hpp>
```

Public Attributes

- std::pair< float, float > **position**
- std::pair< float, float > **size**
- int **rotation**
- std::pair< float, float > **textureSize**
- bool **isHovered**
- int **drawOrder**
- std::string **texturePath**
- std::string **text**
- std::function< void()> **buttonAction**

4.10.1 Detailed Description

Represents a renderable object in the display.

The documentation for this struct was generated from the following file:

- src/Include/Utils.hpp

4.11 Arc::LibraryMetadata Struct Reference

Metadata information extracted from a dynamic library.

`#include <Core.hpp>`

Public Attributes

- std::string **name**
- std::string **path**
- LibraryType **type**

4.11.1 Detailed Description

Metadata information extracted from a dynamic library.

The documentation for this struct was generated from the following file:

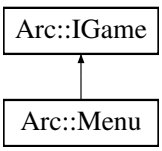
- src/Core/Core.hpp

4.12 Arc::Menu Class Reference

Main menu interface for the Arcade application.

`#include <Menu.hpp>`

Inheritance diagram for Arc::Menu:



Public Member Functions

- Menu (std::map< std::string, Item > &itemList, Core &core, const std::vector< std::pair< std::string, std::↩ string > > &games, const std::vector< std::pair< std::string, std::string > > &renderers)

  Constructs the Menu with game and renderer options.
- ~**Menu** () override

  Destructor.
- void update (Event event) override

  Updates menu state based on events.
- void handleKey (Event event) override

  Handles keyboard input for menu navigation and selection.
- std::vector< int > getScore () const override

  Gets the game score (always 0 for menu).
- bool isGameOver () const override

  Checks if the game is over (always false for menu).
- std::string getUsername () const

  Retrieves the username entered by the player.
- std::string getHighScore (const std::string &gamePath) const

  Retrieves the high score for a specific game.

4.12.1 Detailed Description

Main menu interface for the Arcade application.

The Menu class provides the user interface for selecting games and renderers. It allows users to navigate through available games and rendering backends, enter a username, and launch their selected game.

- **Game Selection**: Users can navigate through available games using LEFT/RIGHT arrows
- **Renderer Selection**: Users can choose rendering backends using UP/DOWN arrows
- **Username Input**: Support for alphanumeric username entry (max 10 characters)
- **Game Launching**: Initiates the selected game with the chosen renderer
- **High Score Display**: Shows the highest score for each game

@inherits IGame

See also

IGame

Core

4.12.2 Constructor & Destructor Documentation

4.12.2.1 Menu()

```
Arc::Menu::Menu (
            std::map< std::string, Item > & itemList,
            Core & core,
            const std::vector< std::pair< std::string, std::string > > & games,
            const std::vector< std::pair< std::string, std::string > > & renderers)
```

Constructs the Menu with game and renderer options.

Parameters

itemList	Reference to the menu items map that will be populated
core	Reference to the Core application controller
games	Vector of available games (pair of name and path)
renderers	Vector of available renderers (pair of name and path)

4.12.3 Member Function Documentation

4.12.3.1 getHighScore()

```
std::string Arc::Menu::getHighScore (
            const std::string & gamePath) const
```

Retrieves the high score for a specific game.

Reads the high score from the save directory for the specified game.

Parameters

gamePath	The game filename to read the score from

Returns

Formatted string with username and score, or "No Score" if not found

- Reads from `./save/{gamePath}.txt`
- Expected format: `username:score`
- Returns "No Score" if file doesn't exist or is malformed

4.12.3.2 getScore()

```
std::vector< int > Arc::Menu::getScore () const  [inline], [override], [virtual]
```

Gets the game score (always 0 for menu).

Returns

A vector containing score of 0

Implements Arc::IGame.

4.12.3.3 getUsername()

```
std::string Arc::Menu::getUsername () const  [inline]
```

Retrieves the username entered by the player.

Returns

The username string (max 10 characters)

4.12.3.4 handleKey()

```
void Arc::Menu::handleKey (
            Event event)  [override], [virtual]
```

Handles keyboard input for menu navigation and selection.

Handles the following inputs:

- **LEFT/RIGHT arrows**: Navigate between games
- **UP/DOWN arrows**: Navigate between renderers
- **A-Z keys**: Enter username characters
- **BACKSPACE**: Delete last username character
- **ENTER**: Launch the selected game
- **ESC**: Quit the application
- **Mouse Click**: Activate buttons

Parameters

event	The keyboard event containing key information

Implements Arc::IGame.

4.12.3.5 isGameOver()

```
bool Arc::Menu::isGameOver () const  [inline], [override], [virtual]
```

Checks if the game is over (always false for menu).

Returns

false

Implements Arc::IGame.

4.12.3.6 update()

```
void Arc::Menu::update (
            Event event)  [override], [virtual]
```

Updates menu state based on events.

Parameters

event	The event to process

Note

Currently unused as handleKey() is the primary event handler

Implements Arc::IGame.

The documentation for this class was generated from the following files:

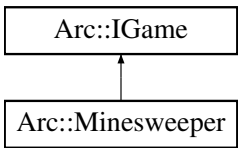
- src/Game/Menu/Menu.hpp
- src/Game/Menu/Menu.cpp

4.13 Arc::Minesweeper Class Reference

Classic Minesweeper game implementation.

`#include <Minesweeper.hpp>`

Inheritance diagram for Arc::Minesweeper:



Public Types

- enum GameState { **DEFEAT** , **VICTORY** , **INPROGRESS** }

  Represents the current state of the game.

Public Member Functions

- Minesweeper (std::map< std::string, Arc::Item > &itemList)

  Constructs a Minesweeper game instance.
- ~**Minesweeper** () override

  Destructor.
- void update (Event event) override

  Updates the game state each frame.
- void handleKey (Event event) override

  Handles mouse input for cell interaction.
- std::vector< int > getScore () const override

  Gets the number of flagged cells (player's score).
- bool isGameOver () const override

  Checks if the game has ended.

4.13.1 Detailed Description

Classic Minesweeper game implementation.

The Minesweeper class implements the classic Minesweeper logic with a 10x12 grid containing 20 randomly placed mines. Players must reveal cells without hitting mines, using numerical clues to safely navigate the board.

- **Grid Size**: 10x12 cells (MINESWEEPER_WIDTH x MINESWEEPER_HEIGHT)
- **Mines**: 20 randomly placed (MINESWEEPER_BOMBS)
- **Game Mechanics**:
  - Left click: Reveal a cell
  - Right click: Flag/unflag a cell as potentially containing a mine
  - Numbers show adjacent mine count
  - Empty cells auto-reveal adjacent safe areas
- **Win Condition**: Reveal all non-mine cells
- **Lose Condition**: Reveal a mine

@inherits IGame

See also

IGame

4.13.2 Constructor & Destructor Documentation

4.13.2.1 Minesweeper()

```
Arc::Minesweeper::Minesweeper (
            std::map< std::string, Arc::Item > & itemList)
```

Constructs a Minesweeper game instance.

Initializes the grid and prepares all UI items. Mines are spawned after the first click to ensure the first move is always safe.

Parameters

itemList	Reference to the item map for rendering UI elements

4.13.3 Member Function Documentation

4.13.3.1 getScore()

```
std::vector< int > Arc::Minesweeper::getScore () const  [inline], [override], [virtual]
```

Gets the number of flagged cells (player's score).

Returns

Vector containing the flag count

Implements Arc::IGame.

4.13.3.2 handleKey()

```
void Arc::Minesweeper::handleKey (
            Event event)  [override], [virtual]
```

Handles mouse input for cell interaction.

Mouse events:

- **Left Click**: Reveal a cell
- **Right Click**: Flag/unflag a cell as a potential mine

Parameters

event	The mouse event containing click position

Implements Arc::IGame.

4.13.3.3 isGameOver()

```
bool Arc::Minesweeper::isGameOver () const  [inline], [override], [virtual]
```

Checks if the game has ended.

Returns

true if game is over (victory or defeat), false if still in progress

Implements Arc::IGame.

4.13.3.4 update()

```
void Arc::Minesweeper::update (
            Event event)  [override], [virtual]
```

Updates the game state each frame.

Processes and detects win/lose conditions.

Parameters

event	The game event (unused)

Implements Arc::IGame.

The documentation for this class was generated from the following files:

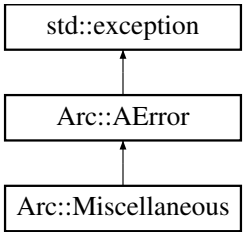
- src/Game/Minesweeper/Minesweeper.hpp
- src/Game/Minesweeper/Minesweeper.cpp

4.14 Arc::Miscellaneous Class Reference

Exception for miscellaneous errors.

`#include <ArcError.hpp>`

Inheritance diagram for Arc::Miscellaneous:



Public Member Functions

- Miscellaneous (std::string message)
  Creates a miscellaneous error.

Public Member Functions inherited from Arc::AError

- AError (std::string message)
  Creates an error with a message.
- const char * what () const noexcept override
  Returns the error message.

4.14.1 Detailed Description

Exception for miscellaneous errors.

4.14.2 Constructor & Destructor Documentation

4.14.2.1 Miscellaneous()

```
Arc::Miscellaneous::Miscellaneous (
            std::string message)  [inline]
```

Creates a miscellaneous error.

Parameters

message	Error description

The documentation for this class was generated from the following file:

- src/Include/ArcError.hpp

4.15 Arc::Mouse Struct Reference

Represents mouse position coordinates.

`#include <Event.hpp>`

Public Attributes

- int **x**
- int **y**

4.15.1 Detailed Description

Represents mouse position coordinates.

The documentation for this struct was generated from the following file:

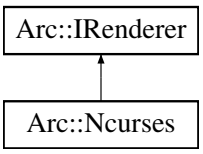
- src/Include/Event.hpp

4.16 Arc::Ncurses Class Reference

Terminal-based renderer using Ncurses library.

`#include <Ncurses.hpp>`

Inheritance diagram for Arc::Ncurses:



Public Member Functions

- Ncurses ()

  Constructs a Ncurses renderer.
- ~Ncurses () override

  Destructor.
- void display (std::map< std::string, Item > &itemList) override

  Renders all items to the terminal.
- bool pollEvent (Event &event) override

  Retrieves and processes terminal input events.
- void createWindow (int width, int height, const std::string &title) override

  Creates and initializes the terminal window.
- void clear ()

  Clears the terminal display.

4.16.1 Detailed Description

Terminal-based renderer using Ncurses library.

The Ncurses class implements a text-based rendering backend that displays the game on a terminal using the Ncurses library. This allows games to run in any terminal environment without graphics dependencies.

- **Display Method**: Character-based terminal rendering
- **Resolution**: Dynamic (default 120x40 characters)
- **Performance**: Lightweight, suitable for remote connections
- **Compatibility**: Works on any terminal with Ncurses support
- **Color Support**: Basic terminal color pairs

@inherits IRenderer

See also

IRenderer

4.16.2 Constructor & Destructor Documentation

4.16.2.1 Ncurses()

```
Arc::Ncurses::Ncurses ()
```

Constructs a Ncurses renderer.

Initializes the Ncurses library and sets up terminal mode. Default dimensions are 120x40 characters with a 1:1 aspect ratio.

4.16.2.2 ~Ncurses()

```
Arc::Ncurses::~Ncurses ()  [override]
```

Destructor.

Cleanly shuts down Ncurses and restores terminal to normal mode.

4.16.3 Member Function Documentation

4.16.3.1 clear()

```
void Arc::Ncurses::clear ()
```

Clears the terminal display.

Removes all content from the terminal, preparing it for the next frame to be rendered.

4.16.3.2 createWindow()

```
void Arc::Ncurses::createWindow (
            int width,
            int height,
            const std::string & title)  [override], [virtual]
```

Creates and initializes the terminal window.

Sets up the terminal with the specified dimensions and title. Configures the rendering area.

Parameters

width	Desired width in characters
height	Desired height in characters
title	Window title (displayed if supported)

Implements Arc::IRenderer.

4.16.3.3 display()

```
void Arc::Ncurses::display (
            std::map< std::string, Item > & itemList)  [override], [virtual]
```

Renders all items to the terminal.

Iterates through the item map and renders each item according to its properties (position, size, text, texture). Displays the complete rendered frame to the terminal.

Parameters

itemList	Reference to the map of items to render

Implements Arc::IRenderer.

4.16.3.4 pollEvent()

```
bool Arc::Ncurses::pollEvent (
            Event & event)  [override], [virtual]
```

Retrieves and processes terminal input events.

Reads keyboard input from the terminal and converts it to Event objects. Handles special keys like arrows, Enter, Escape, etc.

Parameters

event	Reference to the Event object to populate

Returns

true if an event was captured, false otherwise

Implements Arc::IRenderer.

The documentation for this class was generated from the following files:

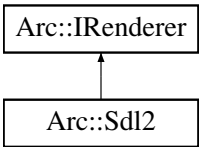
- src/Renderer/Ncurses/Ncurses.hpp
- src/Renderer/Ncurses/Ncurses.cpp

4.17 Arc::Sdl2 Class Reference

Hardware-accelerated graphics renderer using SDL2.

`#include <Sdl2.hpp>`

Inheritance diagram for Arc::Sdl2:



Public Member Functions

- Sdl2 ()
  Constructs an SDL2 renderer.
- ~Sdl2 () override
  Destructor.
- void display (std::map< std::string, Item > &itemList) override
  Renders all items to the SDL2 window.
- bool pollEvent (Event &event) override
  Retrieves and processes input events from SDL2.
- void createWindow (int width, int height, const std::string &title) override
  Creates and initializes the SDL2 window.
- void clear ()
  Clears the renderer (sets background color).

4.17.1 Detailed Description

Hardware-accelerated graphics renderer using SDL2.

The Sdl2 class implements a modern graphics rendering backend using SDL2 (Simple DirectMedia Layer). It provides hardware-accelerated rendering with support for textures, fonts, and complex graphical elements.

- **Rendering**: Hardware-accelerated via SDL2 renderer
- **Font Support**: TrueType fonts (TTF) with size caching
- **Texture Support**: Image loading and caching for performance
- **Cross-platform**: Works on Windows, Linux, macOS
- **Performance**: Optimized with font and texture caching
- **Color Support**: Full RGB color palette

@inherits IRenderer

See also

IRenderer

4.17.2 Constructor & Destructor Documentation

4.17.2.1 Sdl2()

```
Arc::Sdl2::Sdl2 ()
```

Constructs an SDL2 renderer.

Initializes SDL2 libraries but does not yet create a window. Window creation is deferred to createWindow().

4.17.2.2 ~Sdl2()

```
Arc::Sdl2::~Sdl2 ()  [override]
```

Destructor.

Cleans up all SDL2 resources including window, renderer, cached fonts, and cached textures.

4.17.3 Member Function Documentation

4.17.3.1 clear()

```
void Arc::Sdl2::clear ()
```

Clears the renderer (sets background color).

Sets the renderer draw color to black and fills the entire rendering area. Called at the start of each display frame.

4.17.3.2 createWindow()

```
void Arc::Sdl2::createWindow (
            int width,
            int height,
            const std::string & title)  [override], [virtual]
```

Creates and initializes the SDL2 window.

Creates an SDL2 window with the specified dimensions, title, and initializes the renderer with hardware acceleration.

Parameters

width	Window width in pixels
height	Window height in pixels
title	Window title displayed in title bar

Exceptions

AError	if window or renderer creation fails

Implements Arc::IRenderer.

4.17.3.3 display()

```
void Arc::Sdl2::display (
            std::map< std::string, Item > & itemList)  [override], [virtual]
```

Renders all items to the SDL2 window.

Clears the renderer, iterates through items by draw order, renders each item (textures and text), and presents the result.

Parameters

itemList	Reference to the map of items to render

See also

renderItems(), renderItem()

Implements Arc::IRenderer.

4.17.3.4 pollEvent()

```
bool Arc::Sdl2::pollEvent (
            Event & event)  [override], [virtual]
```

Retrieves and processes input events from SDL2.

Polls for SDL2 events and converts them to Event objects. Handles keyboard input, mouse events, and window close events.

Parameters

event	Reference to the Event object to populate

Returns

true if an event was captured, false if no events pending

Implements Arc::IRenderer.

The documentation for this class was generated from the following files:

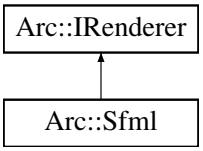
- src/Renderer/Sdl2/Sdl2.hpp
- src/Renderer/Sdl2/Sdl2.cpp

4.18 Arc::Sfml Class Reference

Graphics renderer using SFML (Simple and Fast Multimedia Library).

`#include <Sfml.hpp>`

Inheritance diagram for Arc::Sfml:



Public Member Functions

- Sfml ()
  Constructs an SFML renderer.
- ~Sfml () override
  Destructor.
- void display (std::map< std::string, Item > &itemList) override
  Renders all items to the SFML window.
- bool pollEvent (Event &event) override
  Retrieves and processes input events from SFML.
- void createWindow (int width, int height, const std::string &title) override
  Creates and initializes the SFML render window.
- void clear ()
  Clears the window for the next frame.

4.18.1 Detailed Description

Graphics renderer using SFML (Simple and Fast Multimedia Library).

The Sfml class provides a renderer implementation using SFML, offering a modern object-oriented graphics API with excellent performance and cross-platform compatibility.

- **Rendering**: Fast hardware-accelerated graphics via SFML
- **Font Support**: TrueType fonts with dynamic size caching
- **Texture Support**: Image loading and caching for efficiency
- **Cross-platform**: Windows, Linux, macOS support
- **Performance**: Optimized with size-based font and path-based texture caching
- **Graphics Features**: Sprites, shapes, and text rendering

@inherits IRenderer

See also

IRenderer

4.18.2 Constructor & Destructor Documentation

4.18.2.1 Sfml()

```
Arc::Sfml::Sfml ()
```

Constructs an SFML renderer.

Initializes the SFML library structures but defers window creation to createWindow().

4.18.2.2 ~Sfml()

```
Arc::Sfml::~Sfml ()  [override]
```

Destructor.

Cleans up all SFML resources including the render window, cached fonts, and cached textures.

4.18.3 Member Function Documentation

4.18.3.1 clear()

```
void Arc::Sfml::clear ()
```

Clears the window for the next frame.

Fills the entire window with black color and prepares it for new rendering content.

4.18.3.2 createWindow()

```
void Arc::Sfml::createWindow (
            int width,
            int height,
            const std::string & title)  [override], [virtual]
```

Creates and initializes the SFML render window.

Creates an SFML RenderWindow with specified dimensions and title, ready for rendering.

Parameters

width	Window width in pixels
height	Window height in pixels
title	Window title displayed in the title bar

Exceptions

AError	if window creation fails

Implements Arc::IRenderer.

4.18.3.3 display()

```
void Arc::Sfml::display (
            std::map< std::string, Item > & itemList)  [override], [virtual]
```

Renders all items to the SFML window.

Clears the window, processes all items in draw order, and displays the complete frame.

Parameters

itemList	Reference to the map of items to render

See also

renderItems(), renderItem()

Implements Arc::IRenderer.

4.18.3.4 pollEvent()

```
bool Arc::Sfml::pollEvent (
            Event & event)  [override], [virtual]
```

Retrieves and processes input events from SFML.

Polls for SFML window and input events, converting them to the internal Event format. Handles keyboard, mouse, and window events.

Parameters

event	Reference to the Event object to populate

Returns

true if an event was captured, false if no events pending

Implements Arc::IRenderer.

The documentation for this class was generated from the following files:

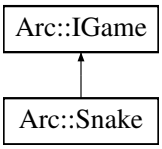
- src/Renderer/Sfml/Sfml.hpp
- src/Renderer/Sfml/Sfml.cpp

4.19 Arc::Snake Class Reference

Classic Snake game implementation.

`#include <Snake.hpp>`

Inheritance diagram for Arc::Snake:



Public Member Functions

- Snake (std::map< std::string, Arc::Item > &itemList)
  Constructs a Snake game instance.
- ~**Snake** () override
  Destructor.
- void update (Event event) override
  Updates the game state each frame.
- void handleKey (Event event) override
  Handles keyboard input for snake control.
- std::vector< int > getScore () const override
  Gets the current game score.
- bool isGameOver () const override
  Checks if the game is over.

4.19.1 Detailed Description

Classic Snake game implementation.

The Snake class implements the classic Snake game mechanics where the player controls a snake that grows by eating food while avoiding collisions with walls and itself.

- **Grid-based gameplay**: 40x40 pixel grid cells
- **Score System**: 1 point per food eaten
- **Speed Control**: Movement delay increases with score
- **Collision Detection**: Walls and self-collision
- **Game Over State**: Detected via collision or manual termination

@inherits IGame

See also

IGame

4.19.2 Constructor & Destructor Documentation

4.19.2.1 Snake()

```
Arc::Snake::Snake (
            std::map< std::string, Arc::Item > & itemList)
```

Constructs a Snake game instance.

Initializes the snake at the center of the play area with one segment, spawns initial food, and prepares all UI items.

Parameters

itemList	Reference to the item map for rendering UI elements

4.19.3 Member Function Documentation

4.19.3.1 getScore()

```
std::vector< int > Arc::Snake::getScore () const  [inline], [override], [virtual]
```

Gets the current game score.

Returns

Vector containing the current score (number of food eaten)

Implements Arc::IGame.

4.19.3.2 handleKey()

```
void Arc::Snake::handleKey (
            Event event)  [override], [virtual]
```

Handles keyboard input for snake control.

Controls the direction of the snake movement:

- **UP arrow**: Move upward (if not moving down)
- **DOWN arrow**: Move downward (if not moving up)
- **LEFT arrow**: Move left (if not moving right)
- **RIGHT arrow**: Move right (if not moving left)

Parameters

event	The keyboard event

Note

Prevents 180-degree turns (snake cannot reverse into itself)

Implements Arc::IGame.

4.19.3.3 isGameOver()

```
bool Arc::Snake::isGameOver () const  [inline], [override], [virtual]
```

Checks if the game is over.

Returns

true if the game is over, false otherwise

Implements Arc::IGame.

4.19.3.4 update()

```
void Arc::Snake::update (
            Event event)  [override], [virtual]
```

Updates the game state each frame.

Processes movement, collision detection, and food consumption. Automatically speeds up as the score increases.

Parameters

event	The game event (unused)

Implements Arc::IGame.

The documentation for this class was generated from the following files:

- src/Game/Snake/Snake.hpp
- src/Game/Snake/Snake.cpp

Chapter 5

File Documentation

5.1 Core.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** Core
00006 */
00007
00008 #pragma once
00009
00010     #include <iostream>
00011     #include <fstream>
00012     #include <sstream>
00013     #include <string>
00014     #include <vector>
00015     #include <memory>
00016     #include <filesystem>
00017     #include <type_traits>
00018     #include <dlfcn.h>
00019     #include <optional>
00020
00021     #include "DLloader.hpp"
00022     #include "IRenderer.hpp"
00023     #include "IGame.hpp"
00024     #include "ArcError.hpp"
00025     #include "Utils.hpp"
00026
00027 namespace Arc
00028 {
00033     struct LibraryMetadata {
00034         std::string name;
00035         std::string path;
00036         LibraryType type;
00037     };
00038
00046     class Core
00047     {
00048         public:
00055             Core(const std::string& defaultRendererLib,
00056                  const std::vector<std::pair<std::string, std::string>& rendererLibs,
00057                  const std::vector<std::pair<std::string, std::string>& gameLibs);
00058
00062             ~Core();
00063
00067             void run();
00068
00074             void launchGame(const std::string& gamePath, const std::string& rendererPath);
00075
00081             static std::optional<LibraryMetadata> extractLibraryMetadata(const std::string& libPath);
00082
00088             static void validateDefaultRenderer(const std::string& rendererPath);
00089
00090             bool _is_running;
00091             int _current_game_index = 0;
00092             int _current_renderer_index = 0;
00093             std::string _username;
00094
00095
```

```
00096         private:
00097             std::unique_ptr<Arc::DLloader<Arc::IRenderer» _current_renderer_loader = nullptr;
00098             std::unique_ptr<Arc::DLloader<Arc::IGame» _current_game_loader = nullptr;
00099
00100             std::unique_ptr<Arc::IRenderer> _current_renderer = nullptr;
00101             std::unique_ptr<Arc::IGame> _current_game = nullptr;
00102
00103             std::vector<std::pair<std::string, std::string» _renderer_libs;
00104             std::vector<std::pair<std::string, std::string» _game_libs;
00105
00106             std::map<std::string, Item> _items;
00107             std::size_t _renderer_index;
00108             std::string _current_game_path;
00109
00114             void loadDefaultRenderer(const std::string& defaultRendererLib);
00115
00120             void loadRenderer(const std::string& rendererPath);
00121
00125             void loadGame();
00126
00127             std::size_t current_score = 0;
00128
00134             std::pair<std::string, std::size_t> getHighScore(const std::string& filename) const;
00135     };
00136 }
```

5.2 Menu.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** Menu
00006 */
00007
00008 #pragma once
00009
00010     #include <iostream>
00011     #include <iostream>
00012     #include <fstream>
00013     #include <sstream>
00014     #include <filesystem>
00015     #include <string>
00016
00017     #include "IGame.hpp"
00018     #include "Core.hpp"
00019     #include "ArcError.hpp"
00020
00021 namespace Arc
00022 {
00042     class Menu : public IGame
00043     {
00044         public:
00053             Menu(std::map<std::string, Item>& itemList, Core& core,
00054                  const std::vector<std::pair<std::string, std::string»& games,
00055                  const std::vector<std::pair<std::string, std::string»& renderers);
00056
00060             ~Menu() override;
00061
00069             void update([[maybe_unused]] Event event) override;
00070
00085             void handleKey(Event event) override;
00086
00091             std::vector<int> getScore() const override { return {0}; };
00092
00097             bool isGameOver() const override { return false; };
00098
00103             std::string getUsername() const { return _username; };
00104
00118             std::string getHighScore(const std::string& gamePath) const;
00119
00120
00121             private:
00122                 std::string _username = "";
00123
00133                 void handleMouseClick(int x, int y);
00134
00143                 bool isPointInRect(int x, int y, const Item& item) const;
00144
00151                 void updateGameItems();
00152
00159                 void updateRendererItems();
00160
```

```
00161                 std::map<std::string, Item>& _items;
00162                 Core& _core;
00163                 const std::vector<std::pair<std::string, std::string>& _games;
00164                 const std::vector<std::pair<std::string, std::string>& _renderers;
00165     };
00166 }
```

5.3 Minesweeper.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** Minesweeper
00006 */
00007
00008 #pragma once
00009
00010     #include "IGame.hpp"
00011     #include "Utils.hpp"
00012
00013     #include <cstdint>
00014     #include <random>
00015     #include <iomanip>
00016     #include <iostream>
00017     #include <queue>
00018
00019     #define MINESWEEPER_WIDTH 10
00020     #define MINESWEEPER_HEIGHT 12
00021     #define MINESWEEPER_BOMBS 20
00022
00023 namespace Arc
00024 {
00047     class Minesweeper : public IGame
00048     {
00049         public:
00054             enum GameState
00055             {
00056                 DEFEAT,
00057                 VICTORY,
00058                 INPROGRESS
00059             };
00060
00069             Minesweeper(std::map<std::string, Arc::Item>& itemList);
00070
00074             ~Minesweeper() override;
00075
00083             void update([[maybe_unused]] Event event) override;
00084
00094             void handleKey(Event event) override;
00095
00100             std::vector<int> getScore() const override { return {_nb_flags}; };
00101
00106             bool isGameOver() const override { return (_game_over) ; }
00107
00108         private:
00113             struct Cell
00114             {
00115                 bool is_bomb = false;
00116                 bool is_revealed = false;
00117                 bool is_flagged = false;
00118                 std::uint8_t adjacent_bombs = 0;
00119             };
00120
00121             std::vector<Cell> _grid;
00122             std::uint8_t _nb_flags;
00123             GameState _game_state;
00124             bool _game_over;
00125             bool _has_started;
00126
00127             std::map<std::string, Item>& _items;
00128
00136             bool isAdjacent(std::size_t c1, std::size_t c2);
00137
00146             void revealEmptyArea(std::size_t index);
00147
00156             void spawnMines(std::size_t first_click);
00157
00164             void detectAdjacentMines();
00165
00171             void clickRight(std::size_t index);
00172
00181             void clickLeft(std::size_t index);
```

```
00182
00186             void updateGrid();
00187
00191             void handleCell();
00192
00200             std::size_t getCellIndexFromMouse(int x, int y);
00201
00205             void display() const;
00206     };
00207 }
```

5.4 Snake.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** Snake Game Header
00006 */
00007
00008 #pragma once
00009
00010 #include "IGame.hpp"
00011 #include <iostream>
00012 #include <map>
00013 #include <deque>
00014 #include <chrono>
00015
00016 namespace Arc
00017 {
00036     class Snake : public IGame
00037     {
00038         public:
00047             Snake(std::map<std::string, Arc::Item>& itemList);
00048
00052             ~Snake() override;
00053
00062             void update(Event event) override;
00063
00076             void handleKey(Event event) override;
00077
00082             std::vector<int> getScore() const override { return {_score}; }
00083             bool isGameOver() const override { return _game_over; }
00084
00085         private:
00092             void reset();
00093
00099             void spawnFood();
00100
00107             void moveSnake();
00108
00119             bool checkCollision(const std::pair<float, float>& pos) const;
00120
00124             void updateItems();
00125
00126             std::map<std::string, Item>& _items;
00127
00128             std::deque<std::pair<float, float» _snake;
00129             std::pair<float, float> _direction;
00130             std::pair<float, float> _nextDirection;
00131             std::pair<float, float> _food;
00132
00133             int _score;
00134             bool _game_over;
00135             bool _game_active;
00136
00137             std::chrono::steady_clock::time_point _lastCheck;
00138             int _moveDelayMs;
00139
00140             const float GRID_WIDTH = 40.0f;
00141             const float GRID_HEIGHT = 30.0f;
00142             const float CELL_SIZE = 20.0f;
00143             const float OFFSET_X = 560.0f;
00144             const float OFFSET_Y = 240.0f;
00145     };
00146 }
```

5.5 ArcError.hpp

```
00001 /*
```

```
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** ArcError
00006 */
00007
00008 #pragma once
00009
00010     #include <exception>
00011     #include <string>
00012
00013     #undef EXIT_FAILURE
00014     #define EXIT_FAILURE 84
00015
00016 namespace Arc
00017 {
00022     class AError : public std::exception
00023     {
00024         public:
00029             AError(std::string message) : _message(message) {};
00030
00035             const char *what() const noexcept override {return _message.c_str();};
00036         private:
00037             std::string _message;
00038     };
00039
00044     class ErrorMinor : public AError
00045     {
00046         public:
00051             ErrorMinor(std::string message) : AError(std::move(message)) {}
00052     };
00053
00058     class ErrorMajor : public AError
00059     {
00060         public:
00065             ErrorMajor(std::string message) : AError(std::move(message)) {}
00066     };
00067
00072     class ErrorCritical : public AError
00073     {
00074         public:
00079             ErrorCritical(std::string message) : AError(std::move(message)) {}
00080     };
00081
00086     class Miscellaneous : public AError
00087     {
00088         public:
00093             Miscellaneous(std::string message) : AError(std::move(message)) {}
00094     };
00095 }
```

5.6 DLloader.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** DLloader
00006 */
00007
00008 #pragma once
00009
00010     #include <dlfcn.h>
00011     #include <iostream>
00012     #include <string>
00013     #include <map>
00014
00015 namespace Arc
00016 {
00017     struct Item;
00018
00028     template <typename T>
00029     class DLloader
00030     {
00031         private:
00032             void *_handle;
00033             T *(*_entryPoint)(std::map<std::string, Item>&);
00034
00035         public:
00040             DLloader(const std::string &libPath) : _handle(nullptr), _entryPoint(nullptr)
00041             {
00042                 _handle = dlopen(libPath.c_str(), RTLD_LAZY);
00043                 if (!_handle) {
```

```
00044                     std::cerr « "[ERROR DLloader] Failed to dlopen: " « dlerror() « std::endl;
00045                     return;
00046                 }
00047
00048                 dlerror();
00049                 _entryPoint = reinterpret_cast<T *(*)(std::map<std::string, Item>&)>(dlsym(_handle,
    "entryPoint"));
00050                 if (dlerror()) {
00051                     std::cerr « "[ERROR DLloader] Could not find entryPoint" « std::endl;
00052                     _entryPoint = nullptr;
00053                 }
00054             }
00055
00062             ~DLloader()
00063             {
00064             }
00065
00071             T *getInstance(std::map<std::string, Item>& itemList)
00072             {
00073                 if (!_entryPoint) {
00074                     std::cerr « "[ERROR DLloader::getInstance] entryPoint is null" « std::endl;
00075                     return nullptr;
00076                 }
00077                 T* instance = _entryPoint(itemList);
00078                 return instance;
00079             }
00080     };
00081 }
```

5.7 Event.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** Event
00006 */
00007
00008 #pragma once
00009
00010 namespace Arc
00011 {
00016     enum Key {
00017         None,
00018         A,
00019         B,
00020         C,
00021         D,
00022         E,
00023         F,
00024         G,
00025         H,
00026         I,
00027         J,
00028         K,
00029         L,
00030         M,
00031         N,
00032         O,
00033         P,
00034         Q,
00035         R,
00036         S,
00037         T,
00038         U,
00039         V,
00040         W,
00041         X,
00042         Y,
00043         Z,
00044         Underscore,
00045         Dash,
00046         Num1,
00047         Num2,
00048         Num3,
00049         Num4,
00050         Num5,
00051         Num6,
00052         Num7,
00053         Num8,
00054         Num9,
00055         Num0,
00056         Tab,
```

```
00057         Left,
00058         Right,
00059         Up,
00060         Down,
00061         Space,
00062         Enter,
00063         Escape,
00064         MouseLeft,
00065         MouseRight,
00066         Backspace,
00067     };
00068
00073     enum Type
00074     {
00075         KeyPressed,
00076         Quit
00077     };
00078
00083     struct Mouse
00084     {
00085         int x;
00086         int y;
00087     };
00088
00093     struct Event
00094     {
00095         Type type;
00096         Key code;
00097         Mouse mouse;
00098     };
00099 }
```

5.8 IGame.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** IGame
00006 */
00007
00008 #pragma once
00009
00010     #include <vector>
00011     #include <string>
00012
00013     #include "Event.hpp"
00014     #include "Utils.hpp"
00015
00016 namespace Arc
00017 {
00025     class IGame
00026     {
00027         public:
00028             virtual ~IGame() = default;
00029
00034             virtual void update([[maybe_unused]] Event event) = 0;
00035
00040             virtual void handleKey(Event event) = 0;
00041
00046             virtual std::vector<int> getScore() const = 0;
00047
00052             virtual bool isGameOver() const = 0;
00053     };
00054 }
```

5.9 IRenderer.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** IRenderer
00006 */
00007
00008 #pragma once
00009
00010     #include <map>
```

```
00011     #include <string>
00012
00013     #include "Event.hpp"
00014     #include "Utils.hpp"
00015
00016 namespace Arc
00017 {
00025     class IRenderer
00026     {
00027         public:
00028             virtual ~IRenderer() = default;
00029
00034             virtual void display(std::map<std::string, Item>& items) = 0;
00035
00042             virtual void createWindow(int width, int height, const std::string& title) = 0;
00043
00049             virtual bool pollEvent(Event& event) = 0;
00050     };
00051 }
```

5.10 Utils.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** Utils
00006 */
00007
00008 #pragma once
00009
00010     #include <iostream>
00011     #include <map>
00012     #include <functional>
00013
00014 namespace Arc
00015 {
00020     enum class LibraryType
00021     {
00022         Game,
00023         Renderer
00024     };
00025
00030     struct Item
00031     {
00032         std::pair<float, float> position;
00033         std::pair<float, float> size;
00034         int rotation;
00035         std::pair<float, float> textureSize;
00036         bool isHovered;
00037         int drawOrder;
00038         std::string texturePath;
00039         std::string text;
00040         std::function<void()> buttonAction;
00041     };
00042 }
```

5.11 Ncurses.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** Ncurses
00006 */
00007
00008 #pragma once
00009
00010     #include "IRenderer.hpp"
00011     #include <ncurses.h>
00012
00013 namespace Arc
00014 {
00033     class Ncurses : public IRenderer
00034     {
00035         public:
00042             Ncurses();
00043
```

```
00049             ~Ncurses() override;
00050
00060             void display(std::map<std::string, Item> &itemList) override;
00061
00071             bool pollEvent(Event& event) override;
00072
00083             void createWindow(int width, int height, const std::string& title) override;
00084
00091             void clear();
00092
00093         private:
00094             float _ratio;
00095             int _maxX;
00096             int _maxY;
00097     };
00098 }
```

5.12 Sdl2.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** Sdl2
00006 */
00007
00008 #pragma once
00009     #include <SDL2/SDL.h>
00010     #include <SDL2/SDL_ttf.h>
00011     #include <SDL2/SDL_image.h>
00012     #include <iostream>
00013     #include <map>
00014     #include <vector>
00015     #include <algorithm>
00016
00017     #include "IRenderer.hpp"
00018     #include "ArcError.hpp"
00019
00020     #define FONT_SIZE 15
00021     #define TEXT_COLOR_R 255
00022     #define TEXT_COLOR_G 255
00023     #define TEXT_COLOR_B 255
00024     #define FONT_PATH "assets/AdwaitaMono-Regular.ttf"
00025
00026 namespace Arc
00027 {
00047     class Sdl2 : public IRenderer
00048     {
00049         public:
00056             Sdl2();
00057
00064             ~Sdl2() override;
00065
00075             void display(std::map<std::string, Item> &itemList) override;
00076
00086             bool pollEvent(Event& event) override;
00087
00100             void createWindow(int width, int height, const std::string& title) override;
00101
00108             void clear();
00109
00110         private:
00111             SDL_Window* _window;
00112             SDL_Renderer* _renderer;
00113             TTF_Font* _font;
00114             std::map<int, TTF_Font*> _font_cache;
00115             std::map<std::string, SDL_Texture*> _texture_cache;
00116
00126             TTF_Font* getFontForSize(int size);
00127
00137             SDL_Texture* getTextureForPath(const std::string& path);
00138
00147             void renderItems(const std::map<std::string, Item>& items);
00148
00157             void renderItem(const std::string& key, const Item& item);
00158
00167             void renderTexture(const Item& item);
00168
00178             void renderText(const Item& item);
00179     };
00180 }
```

5.13 Sfml.hpp

```
00001 /*
00002 ** EPITECH PROJECT, 2026
00003 ** Arcade
00004 ** File description:
00005 ** Sfml
00006 */
00007
00008 #pragma once
00009
00010     #include <SFML/Graphics.hpp>
00011     #include <SFML/Window.hpp>
00012     #include <iostream>
00013     #include <map>
00014     #include <vector>
00015     #include <algorithm>
00016
00017     #include "IRenderer.hpp"
00018     #include "ArcError.hpp"
00019
00020     #define FONT_SIZE 15
00021     #define TEXT_COLOR_R 255
00022     #define TEXT_COLOR_G 255
00023     #define TEXT_COLOR_B 255
00024     #define RATIO 20
00025     #define FONT_PATH "assets/AdwaitaMono-Regular.ttf"
00026
00027 namespace Arc
00028 {
00048     class Sfml : public IRenderer
00049     {
00050         public:
00057             Sfml();
00058
00065             ~Sfml() override;
00066
00076             void display(std::map<std::string, Item> &itemList) override;
00077
00087             bool pollEvent(Event& event) override;
00088
00101             void createWindow(int width, int height, const std::string& title) override;
00102
00109             void clear();
00110
00111         private:
00112             sf::RenderWindow* _window;
00113             sf::Font* _font;
00114             std::map<int, sf::Font*> _font_cache;
00115             std::map<std::string, sf::Texture*> _texture_cache;
00116
00126             sf::Font* getFontForSize(int size);
00127
00137             sf::Texture* getTextureForPath(const std::string& path);
00138
00148             void renderItems(const std::map<std::string, Item>& items);
00149
00159             void renderItem(const std::string& key, const Item& item);
00160
00169             void renderTexture(const Item& item);
00170
00180             void renderText(const Item& item);
00181     };
00182 }
```

Index